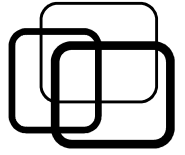


Binary file

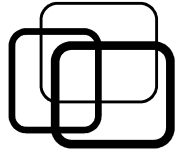
Inst. Nguyễn Minh Huy

Contents



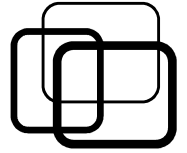
- `main()` arguments.
- Binary file.

Contents



- **main() arguments.**
- **Binary file.**

main() arguments



■ Command-line arguments:

- Program is a giant function!!
- How to pass arguments to program?
- Command-line arguments:
 - Pass arguments to program when calling.
 - main() can get the calling arguments.

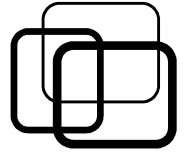
■ Usage:

- Run program in command-line terminal.
- Syntax: **<program> <arg 1> <arg 2> ...**

C:\>BaiTap\baitap1.exe hello 5 /abc

C:\>copy C:\BaiTap\baitap1.exe D:\Files\baitap1.exe

main() arguments



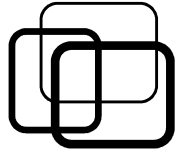
■ main() arguments:

■ Declaration: `int main(int argc, char **argv);`

- `argc`: argument count .
- `argv`: argument variables.
- Arguments passed as strings.
- First argument is program name.

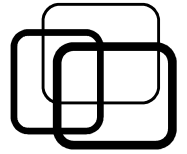
```
int main(int argc, char **argv)  
{  
    cout << "Number of args = " << argc;  
    cout << "Args list:" << endl;  
    for (int i = 0; i < argc; i++)  
        cout << argv[ i ] << endl;  
}
```

Contents



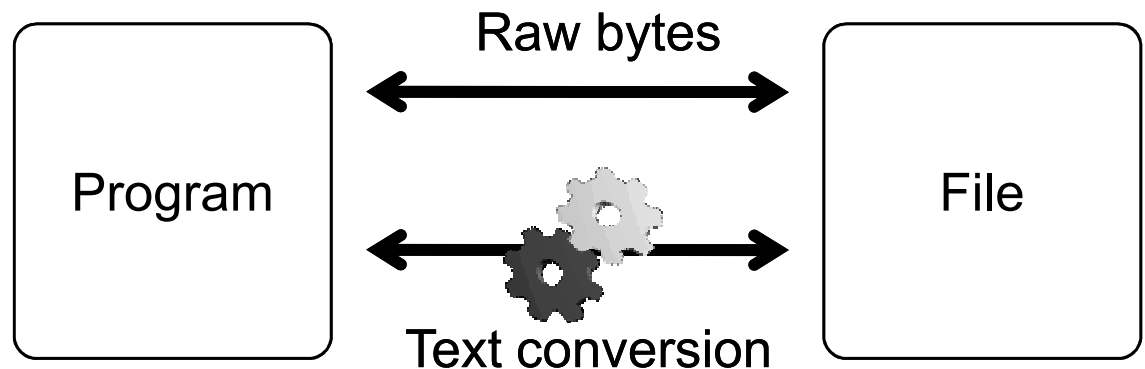
- main() arguments.
- **Binary file.**

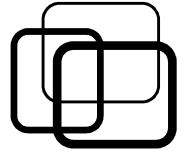
Binary file



■ Binary vs. text mode:

- All file are stored in binary (bit/byte).
- Binary: read/write raw bytes directly.
- Text: read/write through ASCII/Unicode conversion.
- C/C++ binary mode:
 - C: [r / w / a] **b**.
 - C++: [std::ios::in / out / app] | **std::ios::binary**.

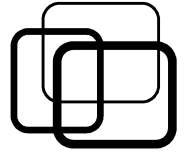




■ Read/write binary in C:

- Transfer data memory pointer $\Leftrightarrow$ file pointer.
- Read/write in blocks of bytes.
 - **fread**(<mem ptr>, <block size>, <block count>, <file ptr>);
 - **fwrite**(<mem ptr>, <block size>, <block count>, <file ptr>);
 - Return: actual read/write blocks.
- ➔ End of file: actual read/write blocks < block count.

```
FILE *f = fopen("C:/data.bin", "rb");
int    x;
char  *p = new char[ 100 ];
if ( f ) {
    fread( &x, sizeof(int), 1, f );    // Read 4 bytes into x.
    fread( p, sizeof(char), 100, f );// Read 100 bytes into p.
    fclose( f );
}
```

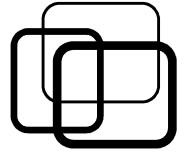


■ Read/write binary in C++:

■ Transfer data memory pointer $\Leftrightarrow$ file stream.

- **<stream>.read(<mem ptr>, <byte count>);**
- **<stream>.write(<mem ptr>, <byte count>);**
- Get actual bytes: **<stream>.gcount(), <stream>.pcount()**.
- ➔ End of file: actual bytes < byte count.

```
std::ifstream f("C:/data.bin", std::ios::binary);
int    x;
char  *p = new char[ 100 ];
if ( f ) {
    f.read( &x, sizeof(int) );           // Read 4 bytes into x.
    f.read( p, sizeof(char) * 100 );// Read 100 bytes into p.
    f.close( );
}
```



■ Move read/write position:

- Only works with opening file.

■ C version:

- **fseek**(<file pointer>, <offset>, <origin>);
- <origin>: SEEK_SET, SEEK_END, SEEK_CUR.
- <offset>: can be negative.

fseek(f, 4, SEEK_SET); // Move to 4 byte from file begin.

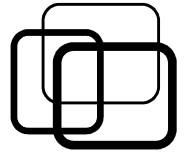
fseek(f, -8, SEEK_END); // Move to 8 byte from file end.

fseek(f, -2, SEEK_CUR); // Move backward 2 byte from current.

■ C++ version:

- **<stream>.seekg**(<offset>, <origin>)
- **<stream>.seekp**(<offset>, <origin>)
- <origin>: std::ios::beg / end / cur.

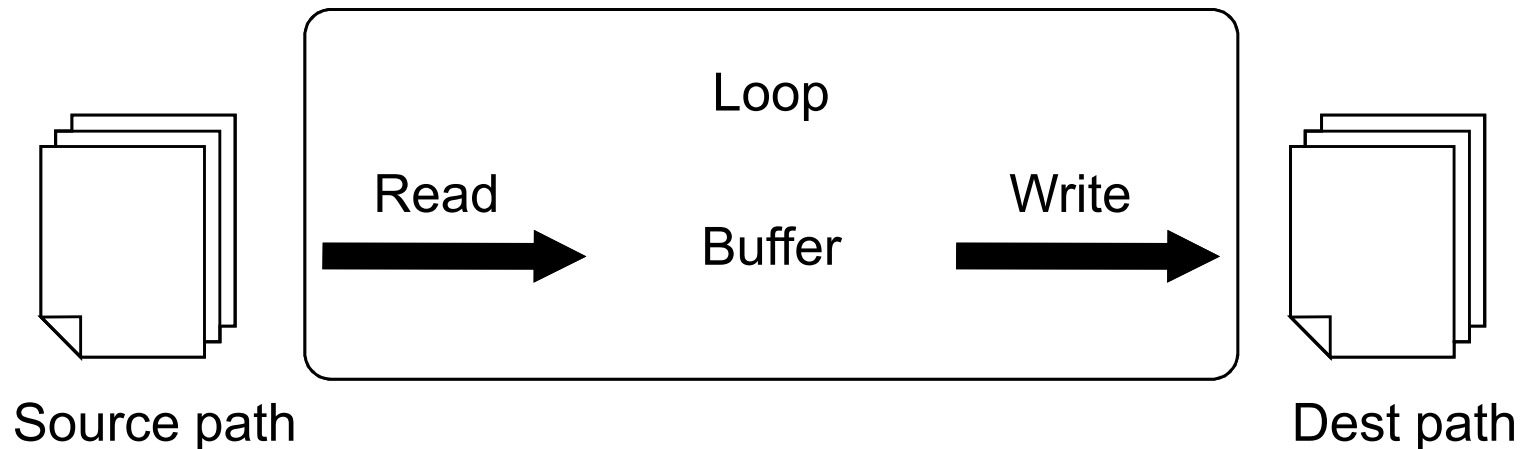
Binary file

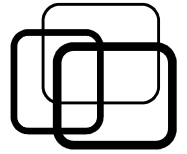


■ Practice:

- Write function to copy file:

bool **copy_file**(<source path>, <dest path>).



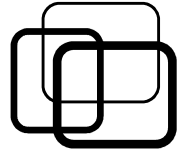


■ Read/write struct:

- Read/write one-by-one bytes of whole struct to file.
 - ➔ More effective than read/write struct members.

```
struct Fraction {  
    int num;  
    int denom;  
};
```

```
void readFraction(FILE *f, Fraction &p) { // Read 8 bytes from file into p.  
    fread( &p, sizeof(Fraction), 1, f ); // First 4 bytes into p.num.  
}                                         // Second 4 bytes into p.denom.  
  
void writeFraction(FILE *f, Fraction p) { // Write 8 bytes p to file.  
    fwrite( &p, sizeof(Fraction), 1, f ); // p.num write to first 4 bytes.  
}                                         // p.denom write to second 4 bytes.
```



■ Library <stdint.h>:

■ What size of int in C?

➔ Depends on platform.

■ Binary file read/write needs fix-sized integer.

➔ Use <stdint.h>

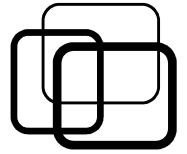
■ Fix-sized integer:

- 1 byte: int8_t, uint8_t.
- 2 bytes: int16_t, uint16_t.
- 4 bytes: int32_t, uint32_t.
- 8 bytes: int64_t, uint64_t.

```
#include <stdint.h>

struct Fraction
{
    int32_t num;
    int32_t denom;
};
```

Summary



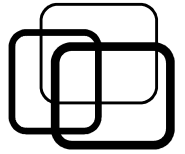
■ main() arguments:

- Calling arguments from command-line terminal.
- Syntax: `int main(int argc, char **argv)`.

■ Binary file:

- Binary mode: read/write directly.
- Text mode: read/write through text conversion.
- C: `fread`, `fwrite`, `fseek`.
- C++: `<stream>.read`, `<stream>.write`, `<stream>.seekg`.
- `<stdint.h>`: fix-sized integers.





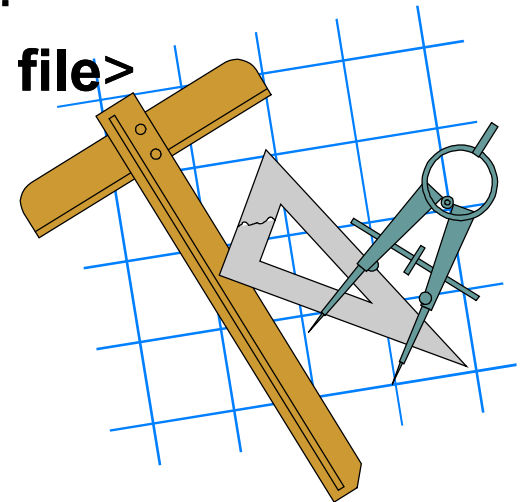
■ Practice 9.1:

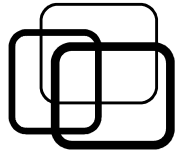
Write C/C++ program named “COPY” to copy file in command line.

Command-line syntax:

- Syntax 1: copy source file to destination file:
`COPY <source file> <destination file>`
- Syntax 2: copy source file to destination path (keep filename):
`COPY <source file> <destination path>/`
- Syntax 3: join file 1 and file 2 to destination file:
`COPY <file 1> + <file 2> <destination file>`
- Syntax 4: show help:
`COPY -?`

Note: use dynamic string and binary file.





■ Practice 9.2 (*):

Write C/C++ program to cut Bitmap file into equal parts in command-line. Each part is saved in a new Bitmap file.

Command-line syntax:

`<program> <Bitmap file> [-h <parts in height>] [-w <parts in width>]`

Example: program cutbmp.exe

- Cut 3 parts in height (save in 3 new Bitmap files):

`cutbmp.exe d:/images/img1.bmp -h 3`

- Cut 2 parts in height, 4 parts in width
(save in 8 new Bitmap files)

`cutbmp.exe d:/images/img1.bmp -h 2 -w 4`

